

Guia de Consulta Rápida: better-sqlite3 no Electron

Curso: Programador de Sistemas • SENAC Centro **Professor:** Jeronimo Silva

Tema: Manipulação de Banco de Dados SQLite com JavaScript

1. VISÃO GERAL: ONDE O BANCO DE DADOS EXECUTA?

Por motivos de segurança, o **acesso direto ao banco de dados SQLite é feito exclusivamente no Processo Principal (main.js)**. O processo Renderer (front-end) nunca acessa o arquivo do banco diretamente; ele faz requisições via ponte IPC (preload.js) para que o main.js execute a query e retorne os resultados.

Importação da Conexão:

No topo do seu arquivo main.js (geralmente em src/main.js), importamos o arquivo de conexão criado na nossa estrutura de pastas (src/database/conexao.js):

```
// Localizado em: src/main.js
const db = require('./database/conexao'); // Importa a conexão com o banco SQLite

// 1. Preparar a instrução SQL (em memória)
const stmt = db.prepare('SELECT * FROM contatos WHERE id = ?');

// 2. Executar passando os parâmetros
const contato = stmt.get(1);
```

2. TABELA RESUMO DOS MÉTODOS DE EXECUÇÃO

Método	Tipo de Query	O que ele recebe?	O que ele retorna?	Exemplo de Uso
stmt.all(...)	Leituras Múltiplas (SELECT)	Parâmetros da query (ex: busca, limite)	Array (Lista) de Objetos. Retorna [] se nada for encontrado.	Listar todos os contatos; buscar por nome com LIKE .
stmt.get(...)	Leitura Única (SELECT)	Parâmetro identificador (ex: ID exato, e-mail exato)	Um único Objeto com as colunas. Retorna undefined se	Buscar contato por E-mail; buscar por ID para detalhar.

Método	Tipo de Query	O que ele recebe?	O que ele retorna?	Exemplo de Uso
			nada for encontrado.	
<code>stmt.run(...)</code>	Escritas (<code>INSERT</code> , <code>UPDATE</code> , <code>DELETE</code>)	Valores a gravar ou ID a deletar/ atualizar	Objeto de Status da Operação com metadados do SQLite.	Adicionar novo contato; deletar registro por ID.

3. ANATOMIA DO RETORNO DE `STMT.RUN()` (ESCRITAS)

Quando rodamos um comando de alteração no banco (como `INSERT` ou `DELETE`), o SQLite não retorna dados da tabela, mas sim um objeto informativo contendo metadados da execução:

```
const result = stmt.run(nome, telefone, email);
// result é um objeto como: { changes: 1, lastInsertRowid: 4 }
```

Propriedades Essenciais de `result`:

1. **`lastInsertRowid`**: Retorna o ID (número inteiro) que o banco de dados atribuiu automaticamente ao novo registro.
 - *Ideal para:* Saber qual foi o ID do contato recém-criado em um `INSERT`.
2. **`changes`**: Retorna o número de linhas que foram modificadas ou afetadas no banco de dados.
 - *Ideal para:* Verificar se um `DELETE` ou `UPDATE` foi bem-sucedido (se `changes > 0`, significa que o registro existia e foi alterado/deletado).

4. BOILERPLATE COPIE & COLE (PADRÃO DE SEGURANÇA E ESTABILIDADE)

Abaixo está o modelo ideal e limpo de código para implementar as operações no processo Main (`main.js`):

Exemplo de `SELECT` Múltiplo (`stmt.all`)

```
ipcMain.handle('obter-contatos', () => {
  try {
    const stmt = db.prepare('SELECT * FROM contatos ORDER BY nome ASC');
    return stmt.all(); // Retorna o Array diretamente
  } catch (error) {
    console.error('Erro no banco:', error.message);
    return []; // Retorna lista vazia em caso de erro (estabilidade)
  }
});
```

```
}  
});
```

Exemplo de SELECT Único (stmt.get)

```
ipcMain.handle('buscar-por-email', (event, email) => {  
  try {  
    const stmt = db.prepare('SELECT * FROM contatos WHERE email = ?');  
    const contato = stmt.get(email);  
    return contato || null; // Retorna o objeto do contato ou null se não existir  
  } catch (error) {  
    console.error('Erro ao buscar:', error.message);  
    return null;  
  }  
});
```

Exemplo de INSERT (stmt.run com lastInsertRowid)

```
ipcMain.handle('inserir-contato', (event, nome, telefone, email) => {  
  try {  
    const stmt = db.prepare('INSERT INTO contatos (nome, telefone, email) VALUES  
      (?, ?, ?)');  
    const result = stmt.run(nome, telefone, email);  
    return result.lastInsertRowid; // Retorna o número ID do novo contato  
  } catch (error) {  
    console.error('Erro ao inserir:', error.message);  
    return null; // Retorna null se falhar (ex: e-mail duplicado)  
  }  
});
```

Exemplo de DELETE (stmt.run com changes)

```
ipcMain.handle('deletar-contato', (event, id) => {  
  try {  
    const stmt = db.prepare('DELETE FROM contatos WHERE id = ?');  
    const result = stmt.run(id);  
    return result.changes; // Retorna a quantidade de registros afetados (0 ou 1)  
  } catch (error) {  
    console.error('Erro ao deletar:', error.message);  
    return 0;  
  }  
});
```



5. APÊNDICE: BOILERPLATE DO ARQUIVO DE CONEXÃO (CONEXAO.JS)

Para que as consultas e escritas do `main.js` funcionem, precisamos inicializar e exportar a conexão ativa do SQLite. Use este boilerplate padrão para o seu arquivo `src/database/conexao.js` :

```
// Localizado em: src/database/conexao.js
const Database = require('better-sqlite3');
const path = require('path');

// 1. Define o caminho físico onde o arquivo de banco (.db) será criado.
// Subindo dois níveis (../..) a partir de 'src/database/' para criar na pasta 'database'
// da raiz do projeto.
const dbPath = path.join(__dirname, '../../database/contatos.db');

// 2. Abre a conexão com o banco de dados.
// O parâmetro 'verbose' faz com que todas as queries SQL sejam logadas no terminal para
// depuração.
const db = new Database(dbPath, { verbose: console.log });

// 3. Executa um comando DDL para criar a tabela automaticamente se ela não existir
db.exec(`
  CREATE TABLE IF NOT EXISTS contatos (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nome TEXT NOT NULL,
    telefone TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL
  )
`);

// 4. Exporta a conexão ativa para que possa ser importada no main.js via 'require()'
module.exports = db;
```